



JASVIR SINGH

Full stack web developer

TECHNICAL SKILLS

Frontend: HTML, CSS, JavaScript, React.js, Bootstrap, Tailwind CSS

Backend: Node.js, Express.js, PHP

Databases: MySQL, MongoDB

Tools: Git, GitHub, VS Code

SOFT SKILLS

Problem-solving, teamwork, time management, Quick learn

CONTACT

PHONE:
+91 8360419086

EMAIL:
Jasvishal123@gmail.com

ADDRESS:
Village – Attalgarh,
Teh – Mukerian,
Distt- Hoshiarpur
Punjab (144211)

PROFESSIONAL SUMMARY

I am an enthusiastic Web Developer Intern with hands-on experience in MERN stack, PHP, MySQL, and basic WordPress development. I have worked on live and academic projects involving responsive website design, database integration, and basic backend functionality.

EDUCATION

- **Master of Computer Applications (MCA)**
DAV University, Jalandhar | **CGPA: 8.77** | 2024 – 2026
- **Bachelor of Computer Science**
DAV University, Jalandhar | **CGPA: 6.88** | 2020 – 2023
- **12th (Non - Medical)**
Govt. Sen. Sec. School, Mukerian | **85.7%** | 2019 – 2020

PROJECTS

- **E-Commerce Platform (DAV University):** Built a full-featured online store with product listings, shopping cart, and secure user authentication using PHP sessions and MySQL for order management.
- **Western Cleaning Service Website:** Designed and developed a responsive service-booking platform with PHP and MySQL.
- **Nationwide Movers & Packers Website:** Delivered a professional business website leveraging PHP, MySQL, and Bootstrap for modern UI.
- **Portfolio Website:** Created a responsive portfolio with React.js and Tailwind CSS to showcase 5+ projects and technical skills.

TRAINING & CERTIFICATIONS

- Frontend Web Development (45 Days) – Ziion Technology, Mohali
- Digital Marketing (45 Days) – TechCadd Institute, Jalandhar
- PHP Development (45 Days) – Microchip Technology Lab, Mukerian

IMPORTANT LINKS

Portfolio - <https://jasvir.vercel.app/>
GitHub - <https://github.com/jasvir-vis/>
LinkedIn – [Jasvir singh](#)

LANGUAGES

Punjabi, Hindi and English

NOISE

The noise which is an unwanted signal affects an image. Removal of noise from an image is called denoising. While the imaging device carrying out its function, there can occurring possibility of variation of brightness or unwanted signal, which is called noise.

Formula:

$$g(x, y) = f(x, y) + n(x, y)$$

TYPES OF NOISE

1. Additive noise

In Spatial Domain

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
```

```
img_float = img.astype(np.float32)
```

```
mean = 0
```

```
sigma = 25
```

```
noise = np.random.normal(mean, sigma, img.shape)
```

```
noisy_img = img_float + noise
```

```
noisy_img = np.clip(noisy_img, 0, 255).astype(np.uint8)
```

```
gaussian_filtered = cv2.GaussianBlur(noisy_img, (5,5), 0)
```

```
median_filtered = cv2.medianBlur(noisy_img, 5)
```

```
cv2.imshow("Original Image", img)

cv2.imshow("Noisy Image (Spatial)", noisy_img)

cv2.imshow("Gaussian Filter Result", gaussian_filtered)

cv2.imshow("Median Filter Result", median_filtered)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT:

Noise added



In Frequency Domain

```
import cv2
```

```
import numpy as np
```

```
img = cv2.imread('jasvir.jpeg', cv2.IMREAD_GRAYSCALE)
```

```
f = np.fft.fft2(img)
```

```
fshift = np.fft.fftshift(f)
```

```
mean = 0
```

```
sigma = 1000
```

```
noise = np.random.normal(mean, sigma, fshift.shape)
```

```
fshift_noisy = fshift + noise
```

```
rows, cols = img.shape
```

```
crow, ccol = rows//2, cols//2
```

```
x = np.linspace(-cols/2, cols/2, cols)
```

```
y = np.linspace(-rows/2, rows/2, rows)
```

```
x, y = np.meshgrid(x, y)
```

```
d = np.sqrt(x**2 + y**2)
```

```
sigma_freq = 30
```

```

gaussian_filter = np.exp(-(d**2)/(2*(sigma_freq**2)))

fshift_filtered = fshift_noisy * gaussian_filter

f_ishift = np.fft.ifftshift(fshift_filtered)

img_back = np.fft.ifft2(f_ishift)

img_back = np.abs(img_back)

img_back = np.clip(img_back, 0, 255).astype(np.uint8)


cv2.imshow("Original Image", img)

cv2.imshow("Frequency Noisy Image", np.log(1 +
np.abs(fshift_noisy)).astype(np.uint8))

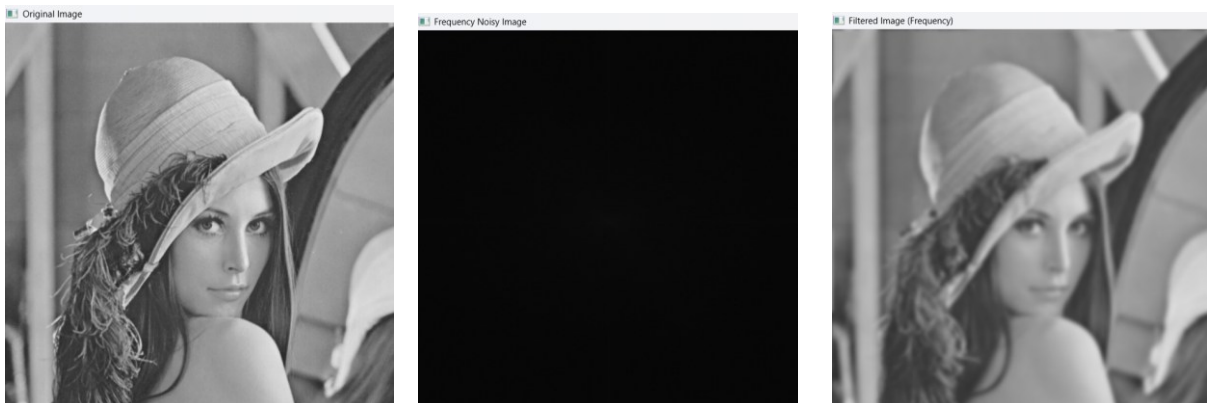
cv2.imshow("Filtered Image (Frequency)", img_back)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

OUTPUT:



2. Impulse noise

In spatial domain

Salt and pepper noise consists of random pixels in an image being set to black or white, resembling the appearance of salt and pepper.

Salt & Pepper noise is defined probabilistically:

$$I_{noisy}(x, y) = \begin{cases} 0 & \text{with probability } p_p \\ 255 & \text{with probability } p_s \\ I(x, y) & \text{with probability } 1 - (p_p + p_s) \end{cases}$$

Program:

```
import cv2

import numpy as np

img = cv2.imread("image.jpg", 0)

noise_prob = 0.05

noisy = img.copy()

salt = np.random.rand(*img.shape) < noise_prob
noisy[salt] = 255

pepper = np.random.rand(*img.shape) < noise_prob
noisy[pepper] = 0

denoised = cv2.medianBlur(noisy, 5)

cv2.imshow("Original", img)

cv2.imshow("Noisy", noisy)

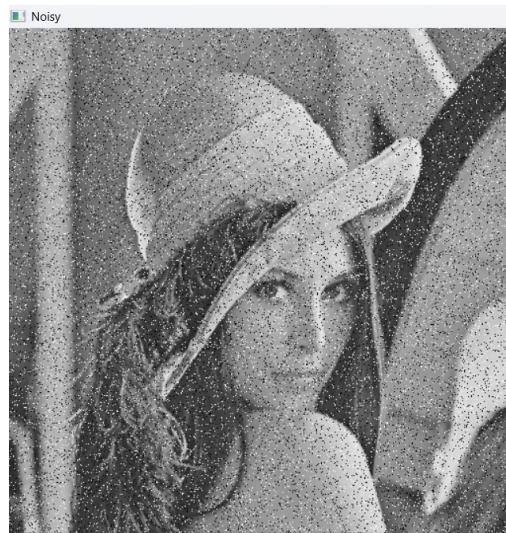
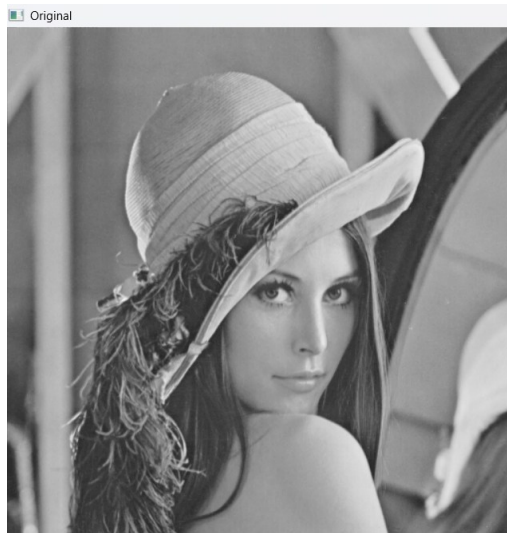
cv2.imshow("Denoised", denoised)
```

```
cv2.waitKey(0)
```

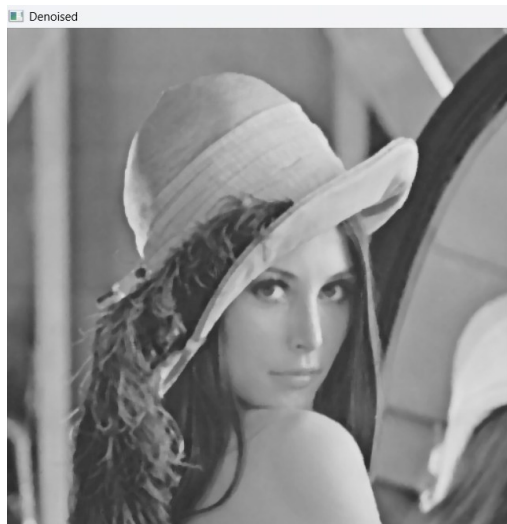
```
cv2.destroyAllWindows()
```

OUTPUT:

Noise Added



Noise Removed



In Frequency domain

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread('lena.png', 0)

def add_salt_pepper_noise(image, prob=0.05):

    noisy = image.copy()

    salt = np.random.rand(*image.shape) < prob

    noisy[salt] = 255

    pepper = np.random.rand(*image.shape) < prob

    noisy[pepper] = 0

    return noisy

noisy_img = add_salt_pepper_noise(img)

f = np.fft.fft2(noisy_img)

fshift = np.fft.fftshift(f)

magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)

rows, cols = noisy_img.shape

crow, ccol = rows // 2, cols // 2

mask = np.zeros((rows, cols), np.uint8)

r = 50

cv2.circle(mask, (ccol, crow), r, 1, -1)

fshift_filtered = fshift * mask

f_ishift = np.fft.ifftshift(fshift_filtered)
```



```
img_back = np.fft.ifft2(f_ishift)
```

```
img_back = np.abs(img_back)
```

```
plt.figure(figsize=(10, 8))
```

```
plt.subplot(2,3,1), plt.imshow(img, cmap='gray'), plt.title("Original")
```

```
plt.subplot(2,3,2), plt.imshow(noisy_img, cmap='gray'), plt.title("Noisy")
```

```
plt.subplot(2,3,3), plt.imshow(magnitude_spectrum, cmap='gray'), plt.title("Spectrum")
```

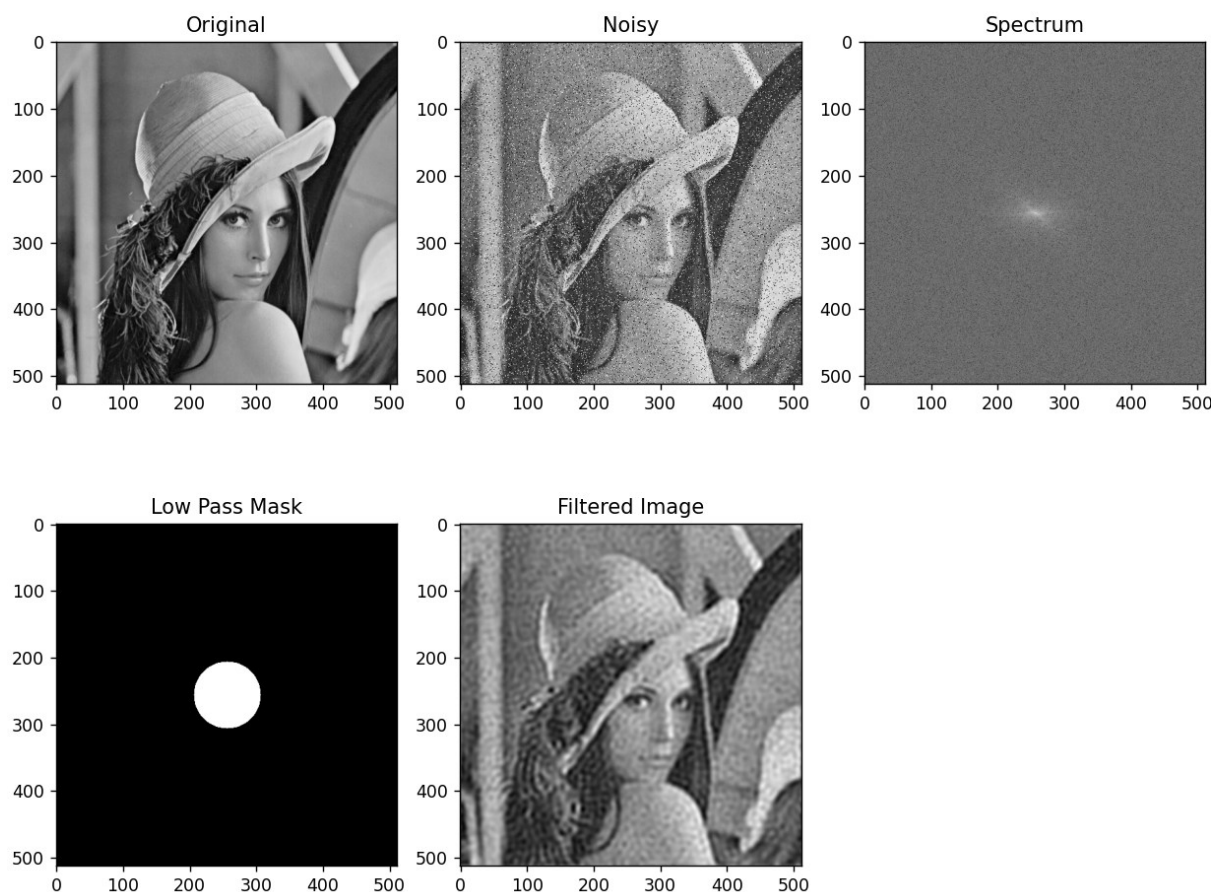
```
plt.subplot(2,3,4), plt.imshow(mask, cmap='gray'), plt.title("Low Pass Mask")
```

```
plt.subplot(2,3,5), plt.imshow(img_back, cmap='gray'), plt.title("Filtered Image")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



3. Multiplicative noise

In Spatial Domain

Program:

```
import cv2

import numpy as np

img = cv2.imread("lena.png", 0) / 255.0

noise = np.random.normal(0, 0.2, img.shape)

noisy = img + img * noise #  $g = I * (1 + n)$ 

log_img = np.log1p(noisy)

filtered = cv2.GaussianBlur(log_img, (5,5), 0)

result = np.expm1(filtered)

result = cv2.normalize(result, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

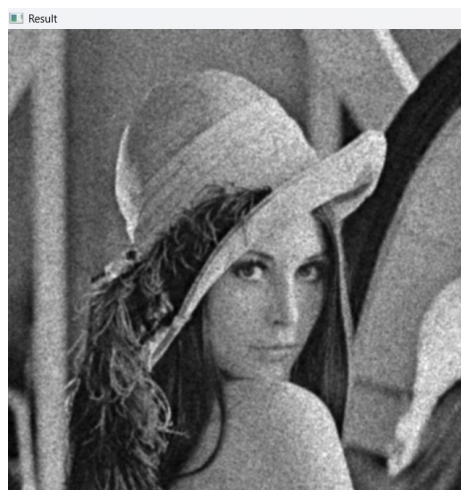
cv2.imshow("Noisy", (noisy*255).astype(np.uint8))

cv2.imshow("Result", result)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT:



In Frequency Domain

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img = cv2.imread('image.jpg', 0)

img = img / 255.0

noise = np.random.normal(0, 0.3, img.shape)

noisy = img + img * noise # g = I(1+n)


log_img = np.log1p(noisy)

f = np.fft.fft2(log_img)

fshift = np.fft.fftshift(f)


magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)

rows, cols = img.shape

crow, ccol = rows//2 , cols//2

mask = np.zeros((rows, cols), np.uint8)

r = 50

cv2.circle(mask, (ccol, crow), r, 1, -1)

fshift_filtered = fshift * mask

f_ishift = np.fft.ifftshift(fshift_filtered)

img_back = np.fft.ifft2(f_ishift)

img_back = np.real(img_back)
```

```
result = np.expm1(img_back)
```

```
result = cv2.normalize(result, None, 0, 1, cv2.NORM_MINMAX)
```

```
plt.figure(figsize=(10,8))
```

```
plt.subplot(2,3,1), plt.imshow(img, cmap='gray'), plt.title("Original")
```

```
plt.subplot(2,3,2), plt.imshow(noisy, cmap='gray'), plt.title("Noisy")
```

```
plt.subplot(2,3,3), plt.imshow(magnitude_spectrum, cmap='gray'), plt.title("Spectrum")
```

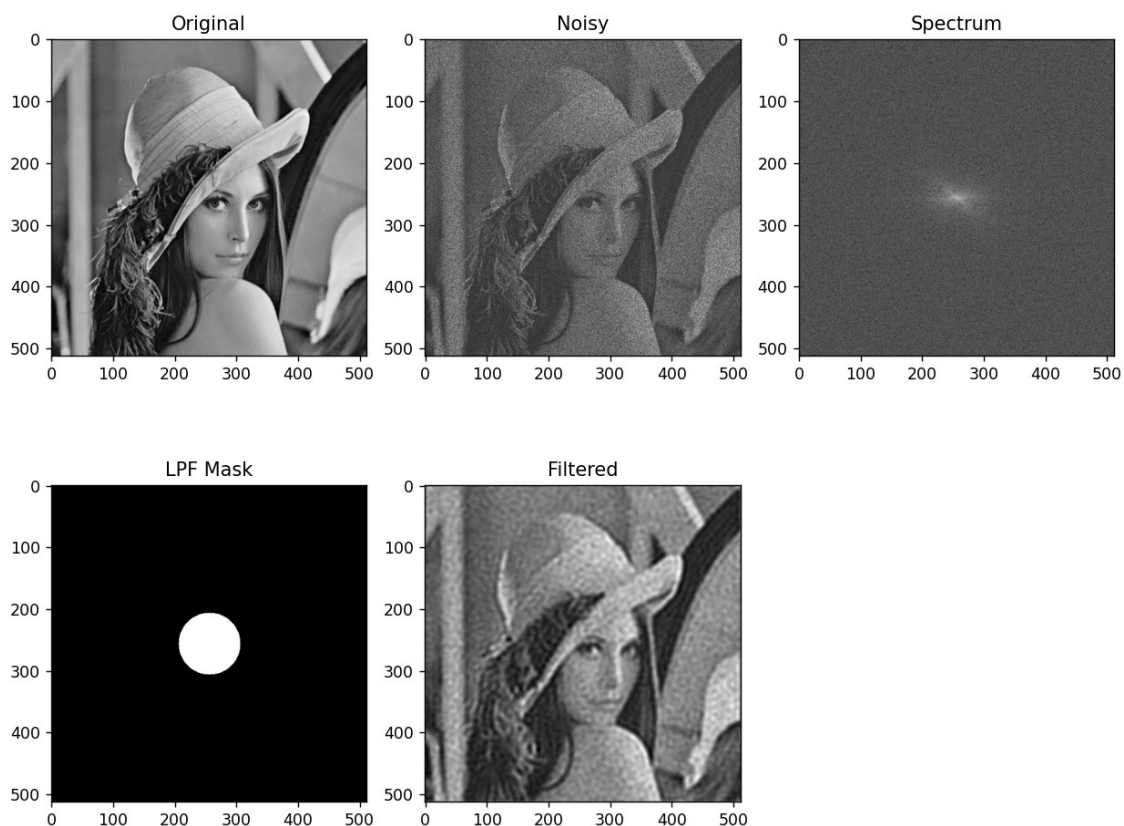
```
plt.subplot(2,3,4), plt.imshow(mask, cmap='gray'), plt.title("LPF Mask")
```

```
plt.subplot(2,3,5), plt.imshow(result, cmap='gray'), plt.title("Filtered")
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



4. Poisson Noise

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread('lena.png', 0)

img = img / 255.0

noisy = np.random.poisson(img * 255) / 255.0

denoised = cv2.GaussianBlur(noisy, (5,5), 0)

plt.figure(figsize=(10,5))

plt.subplot(1,3,1)

plt.imshow(img, cmap='gray')

plt.title("Original")

plt.subplot(1,3,2)

plt.imshow(noisy, cmap='gray')

plt.title("Poisson Noise")

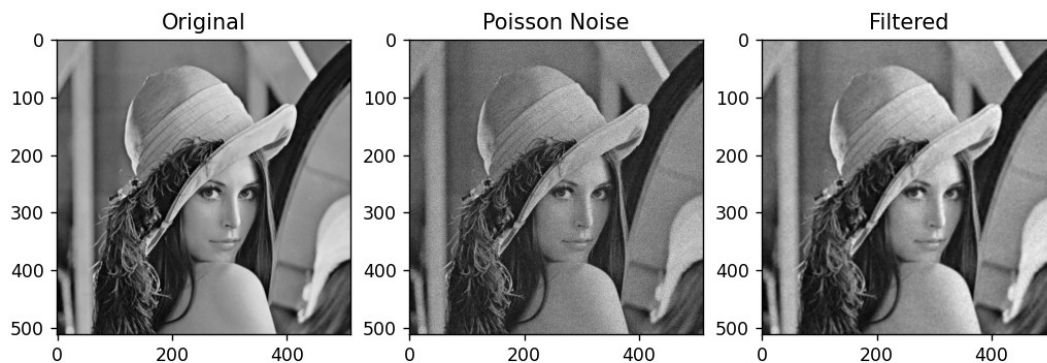
plt.subplot(1,3,3)

plt.imshow(denoised, cmap='gray')

plt.title("Filtered")

plt.show()
```

OUTPUT:



In Frequency Domain

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)
```

```
img_norm = img / 255.0
```

```
scaled = img_norm * 255
```

```
poisson_noise = np.random.poisson(scaled).astype(np.float32)
```

```
poisson_noise = poisson_noise / 255.0
```

```
f = np.fft.fft2(poisson_noise)
```

```
fshift = np.fft.fftshift(f)
```

```
magnitude = 20 * np.log(np.abs(fshift) + 1)
```

```
f_ishift = np.fft.ifftshift(fshift)
```

```
img_back = np.fft.ifft2(f_ishift)
```

```
img_back = np.abs(img_back)
```

```
plt.figure(figsize=(10,8))
```

```
plt.subplot(2,2,1)
```

```
plt.imshow(img_norm, cmap='gray')
```

```
plt.title("Original Image")
```

```
plt.axis('off')
```

```
plt.subplot(2,2,2)
```

```
plt.imshow(poisson_noise, cmap='gray')
```

```
plt.title("Poisson Noise Image")
```

```
plt.axis('off')
```

```
plt.subplot(2,2,3)
```

```
plt.imshow(magnitude, cmap='gray')
```

```
plt.title("FFT Spectrum")
```

```
plt.axis('off')
```

```
plt.subplot(2,2,4)
```

```
plt.imshow(img_back, cmap='gray')  
  
plt.title("Reconstructed (Inverse FFT)")  
  
plt.axis('off')
```

```
plt.tight_layout()
```

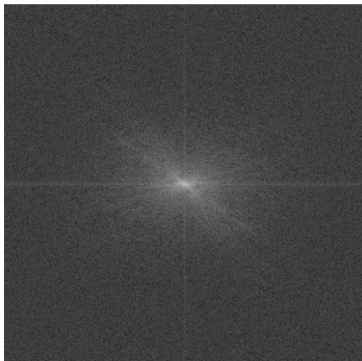
```
plt.show()
```

OUTPUT:

Original Image



FFT Spectrum



Poisson Noise Image



Reconstructed (Inverse FFT)



Parsing in NLP

Introduction to Parsing

Parsing is an important technique in **Natural Language Processing (NLP)** used to analyze the **grammatical structure of a sentence**.

When humans read a sentence, we easily understand **who is doing the action and what the action is**.

However, computers cannot understand natural language directly. Parsing helps the computer **break a sentence into smaller parts and understand relationships between words**.

In simple words:

Parsing = Analyzing a sentence to identify its grammatical structure.

Example sentence:

“The boy plays football.”

Parsing identifies:

- Subject → *boy*
- Verb → *plays*
- Object → *football*

So, the system understands the meaning of the sentence.

Concept of Parsing

The main concept of parsing is to **convert a sentence into a structured representation**.

A sentence usually contains:

- **Noun Phrase (NP)** – person or thing
- **Verb Phrase (VP)** – action
- **Prepositional Phrase (PP)** – location or relation

Example sentence:

“The girl reads a book.”

Structure: Sentence (S)

- Noun Phrase (NP) → The girl
- Verb Phrase (VP) → reads a book

This structure is usually represented using a **Parse Tree**.

How Parsing Works in NLP

Parsing normally follows these steps.

Step 1 – Input Sentence

Example:

“Students study NLP.”

Step 2 – Tokenization

Sentence is split into words.

Students | study | NLP

Step 3 – Part-of-Speech Tagging

Each word gets a grammatical tag.

Students → Noun

study → Verb

NLP → Noun

Step 4 – Build Parse Structure

The parser organizes the words according to grammar rules.

Sentence

- Subject → Students
- Verb → study
- Object → NLP

Example:

Question: Given Sentence - “the big dog chased the cat”

Grammar Rules

$S \rightarrow NP VP$

$NP \rightarrow Det N \mid Det Adj N$

$VP \rightarrow V NP \mid V$

$Det \rightarrow the$

$Adj \rightarrow big$

$N \rightarrow dog \mid cat$

$V \rightarrow chased$

Answer:

1. Sentence Breakdown

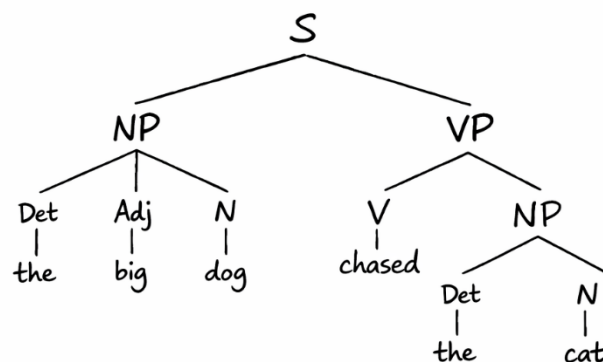
The | big | dog | chased | the | cat

2. Derivation

1. $S \Rightarrow NP VP$
2. $S \Rightarrow Det Adj N VP$
3. $S \Rightarrow the big dog VP$
4. $S \Rightarrow the big dog V NP$
5. $S \Rightarrow the big dog chased NP$
6. $S \Rightarrow the big dog chased Det N$
7. $S \Rightarrow the big dog chased the cat$

3. Final Derived Sentence

the big dog chased the cat



“the big dog chased the cat”

Applications of Parsing

1. **Machine Translation** – helps translate sentences correctly by understanding grammar.
2. **Information Extraction** – identifies important data like subject, object, names, etc.
3. **Question Answering Systems** – understands questions to give accurate answers.
4. **Speech Recognition** – helps convert and interpret spoken language correctly.